

Lecture 20 – Autoencoders and t-SNE

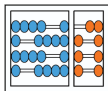
[Various sources; references in the last slide]

MC886 – Machine Learning

Marcelo S. Reis [†]

[†] Departamento de Sistemas de Informação
Instituto de Computação, Universidade Estadual de Campinas

Campinas, May 13, 2026



**Instituto de
Computação**

UNIVERSIDADE ESTADUAL DE CAMPINAS



UNICAMP

Today's summary

Nonlinear dimensionality reduction

Autoencoders: basic principles and algorithm

t-SNE: basic principles and algorithm

Comparison between PCA, autoencoders and t-SNE

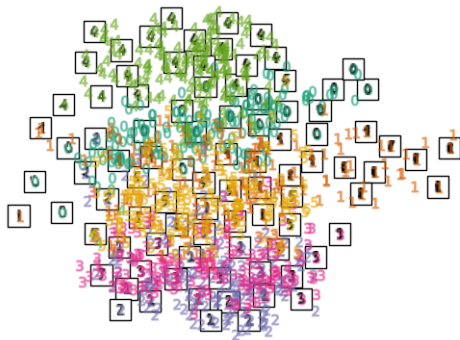
Dimensionality reduction

- ❖ One important application: data visualization
- ❖ Reduction of high-dimensional points to 2D or 3D ones
- ❖ We already learned PCA, a linear projection of the data

Digits dataset (first 6 digits), $x = (x_1, x_2, \dots, x_{64})^T$

0	1	2	3	4	5	0	1	2	3
4	5	0	1	2	3	4	5	0	5
5	5	0	4	1	3	5	1	0	0
2	2	2	0	1	2	3	3	3	3
4	4	1	5	0	5	2	2	0	0
1	3	2	1	4	3	1	3	1	4
3	1	4	0	5	3	1	5	4	4
2	2	2	5	5	4	4	0	0	1
2	3	4	5	0	1	2	3	4	5
0	1	2	3	4	5	0	5	5	5

PCA on **Digits** data (1st and 2nd principal components)



Shortcomings of PCA for data visualization

- ❖ Nonlinear structure in data are overlooked
- ❖ Such nonlinearities might be relevant for a more suitable grouping of points that are similar in the high-dimensional representation (and separation of dissimilar points)
- ❖ Two popular nonlinear dimensionality reduction techniques (also known as *manifold learning*):
 1. Autoencoders
 2. t-distributed Stochastic Neighbor Embedding (t-SNE)

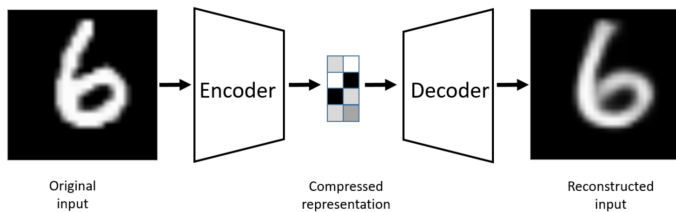
Autoencoders: basic principles

- ❖ A neural network trained in an unsupervised fashion to reconstruct its input
- ❖ **Purpose:** to learn meaningful representation from data
- ❖ More formally, the problem is to learn functions $A: \mathbb{R}^p \rightarrow \mathbb{R}^M$ (encoder) and $B: \mathbb{R}^M \rightarrow \mathbb{R}^p$ (decoder) such that:

$$\arg \min_{A,B} E[\Delta(x, B \circ A(x))],$$

where Δ is a loss (e.g., squared error) and E is the expectation over the distribution of x

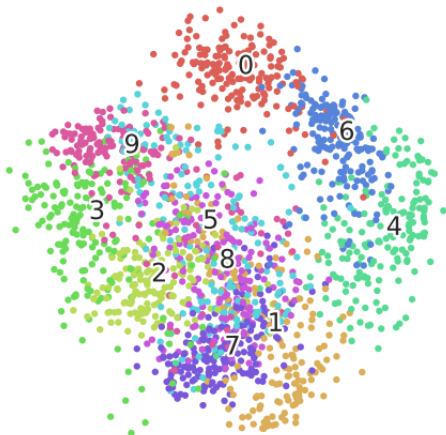
An autoencoder example



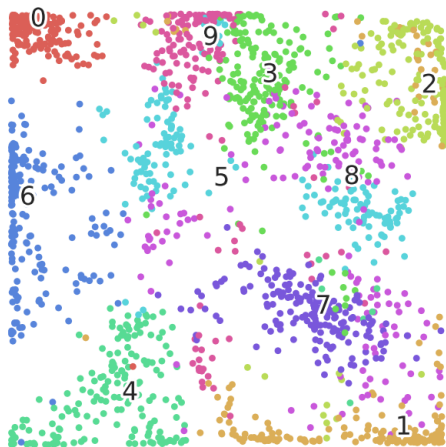
More on autoencoder

- ❖ We define it as a neural network (e.g., *feedforward*), with one or more hidden layers
- ❖ Often we are interested in the representation yielded by the encoder output, whose dimension M is smaller than p
 - We call the compressed output of the encoder as “latent representation”
- ❖ In general, we put a nonlinear activation in the units (e.g., $\tanh$)
 - Otherwise we have a linear autoencoder, which yields a latent similar to the PCA
 - Autoencoder can be seen as a generalization of PCA!

PCA on Digits 10-digit data (1st and 2nd PCs)



Autoencoder on Digits 10-digit data (2D latent vectors)

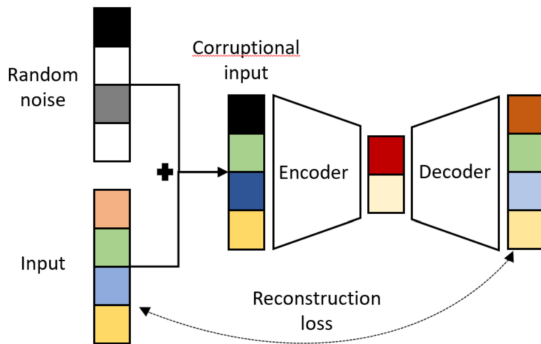


Autoencoder implemented using a feedforward with an input layer (64 units) and 2 hidden layers ($L_1 = 64$ and $L_2 = 2$). Output layer (decoder) had 64 units. All hidden and output units used tanh as activation function

Learning an autoencoder

- ❖ Our dataset is composed of pairs $(x_1, x_1), (x_2, x_2), \dots, (x_n, x_n)$
- ❖ We train the autoencoder using gradient descent and backpropagation algorithms
- ❖ After training, one can use the encoder to get an smaller (and meaningful) data representation
- ❖ Noise can be added during the training, either as a means of regularization or to create a *denoising autoencoder*
 - This latter is used to clean noisy inputs (e.g., images); for this purpose, the decoder is kept for testing

A denoising autoencoder example



The disrupted input image is encoded to a representation and then decoded. The loss is used to reconstruct the original image

t-SNE: basic principles

- ❖ It embeds each p -dimensional point into a 2D or 3D one such that similar points are close together and dissimilar points are far apart by a high probability
- ❖ It works in two stages:
 1. It constructs a probability distribution over point pairs in the p -dimensional space in a way that similar (dissimilar) pairs are assigned a higher (lower) probability
 2. It defines a probability distribution over point pairs in the 2D or 3D-dimensional space and tries to minimize the divergence between this distribution and the p -dimensional one

Initial definitions

- ❖ Our dataset is composed of $x_1, \dots, x_n$, where $x_i = \begin{pmatrix} x_{i1} \\ \vdots \\ x_{ip} \end{pmatrix}$
- ❖ The **map dataset** consists in $z_1, \dots, z_n$, where $z_i \in \mathbb{R}^2$ (or $\mathbb{R}^3$)
- ❖ Every point z_i represents a unique x_i

Stage 1: Construction of distribution in original space

- ❖ We start the construction of the probability distribution in the p -dimensional space by computing the conditional similarity between x_i and x_j :

$$p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|x_i - x_k\|^2 / 2\sigma_i^2)},$$

where $p_{i|i}$ is set to zero and $\sum_j p_{j|i} = 1$ for all i

- ❖ This measures how close x_j is from x_i considers a Gaussian distribution centered in x_i with variance σ_i^2

Choice of σ_i^2

- ❖ σ_i^2 is chosen such that if it is in a dense area (in terms of observations x_k) it receives a smaller variance than points in sparser areas
- ❖ Original t-SNE paper proposes the usage of **perplexity**

Choice of σ_i^2 using perplexity

- ❖ **Perplexity** is roughly a smooth measure of effective number of neighbors
- ❖ For a given datapoint i with probability distribution P_i , the perplexity is defined as:

$$\text{Perp}(P_i) = 2^{H(P_i)},$$

where $H(P_i) = -\sum_j p_{j|i} \log_2 p_{j|i}$ is the Shannon entropy of P_i

- ❖ Binary search is carried out on σ_i^2 values, in order to find the perplexity value defined by user (generally between 5 and 50)

Similarity distribution in original space

- ❖ The final similarity probability distribution in the d -dimensional space is given by:

$$p_{i,j} = \frac{1}{n} \frac{p_{j|i} + p_{i|j}}{2},$$

- ❖ where $p_{i,i} = 0$, $p_{i,j} = p_{j,i}$ and $\sum_{i,j} p_{i,j} = 1$

Stage 2: Definition of 2D distribution and optimization

- ❖ The similarity probability distribution on the points of the map dataset is defined as:

$$q_{i,j} = \frac{(1 + \|z_i - z_j\|^2)^{-1}}{\sum_k \sum_{\ell \neq k} (1 + \|z_k - z_\ell\|^2)^{-1}},$$

where $q_{i,i}$ is set to zero

- ❖ This means that a heavy-tailed **Student t-distribution** with one degree of freedom is used to measure similarities between points in the map dataset
- ❖ Using this distribution instead the Gaussian allows dissimilar points to be placed far apart in the map

Computing $z_1, \dots, z_n$

- ❖ Stage 1 yielded the original dataset distribution $(p_{i,j})$
- ❖ Map dataset distribution $(q_{i,j})$ must be computed in a way that it is as close as possible to $(p_{i,j})$
- ❖ A way to measure similarity between distributions: the **Kullback–Leiber divergence**

Kullback–Leiber divergence

- ❖ The Kullback–Leiber divergence of P from Q is given by:

$$KL(P \parallel Q) = \sum_{i,j} p_{i,j} \log \frac{p_{i,j}}{q_{i,j}}$$

- ❖ It measures how distribution P is different from the reference probability distribution Q
- ❖ KL is our objective function, which we want to minimize with respect to z_i (which we use to compute $q_{i,j}$)
 - To this end, we'll use the old and good gradient descent!

Minimizing the KL objective function

- ❖ The gradient of KL with respect to z_i is given by:

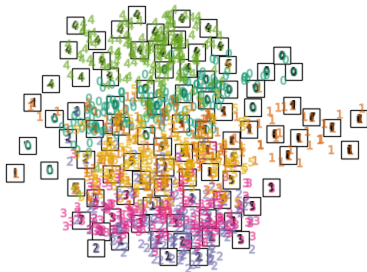
$$\frac{\partial KL(P \parallel Q)}{\partial z_i} = \sum_j (p_{i,j} - q_{i,j})(z_i - z_j)(1 + \|x_i - x_j\|^2)^{-1}$$

- ❖ z_i values are randomly initialized with small values (e.g., sampling from $\mathcal{N}(0, 10^{-4})$)
- ❖ Using the initialization values, $\frac{\partial KL(P \parallel Q)}{\partial z_i}$ and a learning rate α , we run gradient descent, at each iteration updating $z_1, \dots, z_n$
- ❖ **Remark:** $KL(P \parallel Q)$ is not convex

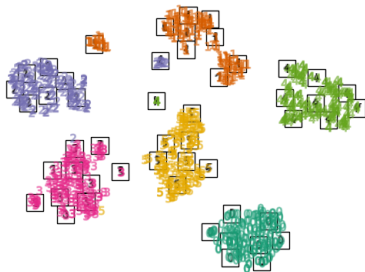
Computational remarks

- ❖ GD execution $\implies$ Depends on α and # steps
- ❖ Pairwise comparisons among n points $\implies O(n^2)$
- ❖ Approximations of the gradient (mini-batches) might be used to speed up (or even turn feasible) the execution
(sklearn implements one of such approximations, and also offers the exact solution)

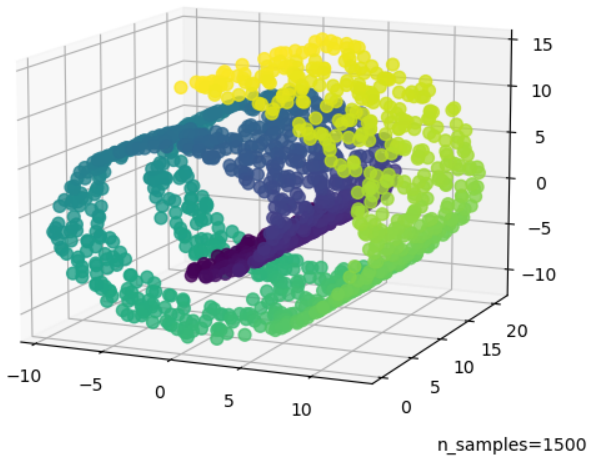
PCA on **Digits** data (1st and 2nd principal components)



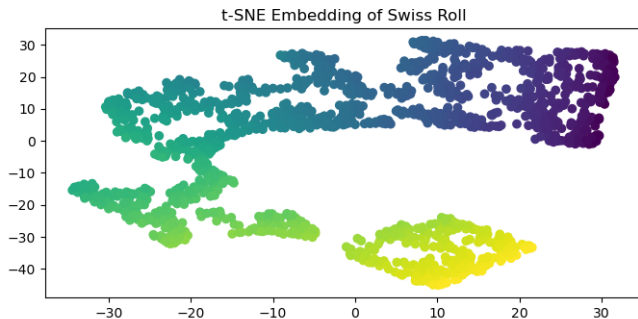
t-SNE on Digits data (2-dimensional map dataset)



Another example: Swiss roll



Another example: Swiss roll



Comparison between PCA, autoencoders and t-SNE

	PCA	Autoencoder	t-SNE
Embedding interpretability	☹️	😊	😊
Computational complexity	😊	😊/☹️	☹️
Deterministic?	Yes	No	No

- ❖ This comparison often (not always) holds for linear vs. nonlinear methods in general

References

- ❖ Bank et al. (2021) “Autoencoders”: arxiv.org/pdf/2003.05991
- ❖ Introduction to the t-SNE algorithm: www.oreilly.com/content/an-illustrated-introduction-to-the-t-sne-algorithm
- ❖ der Maaten and Hinton (2008) “Visualizing Data using t-SNE”: www.jmlr.org/papers/volume9/vandermaaten08a/vandermaaten08a.pdf
- ❖ sklearn documentation: scikit-learn.org